

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – DATA CLEANING CHALLENGE

Course Code:	DSA0502	Course Title:	Query Processing for Data Science
CO Assessed:	CO3	Assessment:	Tool 2 – Data Cleaning Challenge
Weightage:		Total Marks:	
CO3 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates, outliers, formatting issues, and applying techniques like fuzzy matching and regex. (BL5 and BL6).			
Student Name:	Sania Herbert	Reg. No.:	192324062
Section / Batch:	2023/AI&DS	Date of Submission:	10.08.26
Faculty In-charge:	Dr. B.Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I Sania Herbert (192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

Anomaly Detection (5 Marks)	Cleaning Methods (5 Marks)	Data Transformation (5 Marks)	Analytical Comparison (5 Marks)	Governance Discussion (5 Marks)	Total (25 Marks)

Question 1: E-Commerce Customer Transactions Dataset

An online retail company has collected customer transaction data from multiple sales channels. The management wants to identify the top-performing customer segments and products. However, the dataset contains several quality issues.

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City
C101	Raj Kumar	Male	28	Electronics	2	15000 INR	10-01-2025	Chennai
C102	Priya S	Female	35	Fashion	3	1200 INR	11-01-2025	chennai
C103	Arun V	Male	150	Electronics	1	300 USD	12-01-2025	CHENNAI
C104	Kavya R	Female	0	Grocery	-2	800 INR	15-01-2025	Coimbatore
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
C105	Rahul P	Male	42	Fashion	5	2500 INR	18-01-2025	Madurai
NULL	Sneha M	Female	30	Electronics	2	450 USD	05-02-2025	Trichy
C107	Deepak N	Male		Grocery	3	600 INR	07-02-2025	Tiruchirappalli
C108	Meena K	Female	27	Fashion	1	1800 INR	2025-13-01	Salem
C109	John D	Male	31	Electronics	4	250 EUR	15-02-2025	Chennai

Data Issues

- Missing Customer IDs
- Duplicate transactions
- Negative quantities
- Unit prices recorded in different currencies
- Age values such as 0, 150, and blanks
- Inconsistent city names (e.g., Chennai, chennai, CHENNAI)
- Incorrect purchase dates

Tasks

1. Identify and classify all data quality issues. (5 Marks)
2. Design a cleaning strategy for each issue. (5 Marks)
3. Perform data standardization and transformation. (5 Marks)
4. Analyze how cleaning affects total revenue calculations. (5 Marks)
5. Generate a report discussing business risks if the data were not cleaned. (5 Marks)

```

1  import pandas as pd
2  import numpy as np
3  # =====
4  # E-COMMERCE CUSTOMER TRANSACTIONS DATASET
5  # =====
6  # Create the dataset
7  data = {
8      "CustomerID": [
9          "C101", "C102", "C103", "C104", "C105",
10         "C105", None, "C107", "C108", "C109"
11     ],
12     "CustomerName": [
13         "Raj Kumar", "Priya S", "Arun V", "Kavya R", "Rahul P",
14         "Rahul P", "Sneha M", "Deepak N", "Meena K", "John D"
15     ],
16     "Gender": [
17         "Male", "Female", "Male", "Female", "Male",
18         "Male", "Female", "Male", "Female", "Male"
19     ],
20     "Age": [
21         28, 35, 150, 0, 42,
22         42, 30, np.nan, 27, 31
23     ],
24     "ProductCategory": [
25         "Electronics", "Fashion", "Electronics", "Grocery", "Fashion",
26         "Fashion", "Electronics", "Grocery", "Fashion", "Electronics"
27     ],
28     "Quantity": [
29         5, 2, 3, 1, -2, 5,
30         5, 2, 3, 1, 4
31     ],
32     "UnitPrice": [
33         "15000 INR", "1200 INR", "300 USD", "800 INR", "2500 INR",
34         "2500 INR", "450 USD", "600 INR", "1800 INR", "250 EUR"
35     ],
36     "PurchaseDate": [
37         "10-01-2025", "11-01-2025", "12-01-2025", "15-01-2025",
38         "18-01-2025", "18-01-2025", "05-02-2025", "07-02-2025",
39         "2025-13-01", "15-02-2025"
40     ],
41     "City": [
42         "Chennai", "chennai", "CHENNAI", "Coimbatore", "Madurai",
43         "Madurai", "Trichy", "Tiruchirappalli", "Salem", "Chennai"
44     ]
45 }
46 df = pd.DataFrame(data)
47 print("ORIGINAL DATA")
48 print(df)
49 # =====
50 # TASK 1: Identify and classify all data quality issues.
51 # =====
52 print("\n\nTASK 1: DATA QUALITY ISSUES")
53 # Check missing values
54 print("\nMissing Values:")
55 print(df.isnull().sum())
56 # Check duplicate transactions
57 print("\nDuplicate Transactions:")
58 print(df[df.duplicated(keep=False)])

```

```

59 # Check invalid ages
60 print("\nInvalid Ages:")
61 print(df[(df["Age"] < 1) | (df["Age"] > 120)])
62 # Check negative quantities
63 print("\nNegative Quantities:")
64 print(df[df["Quantity"] < 0])
65 # Identify currencies
66 print("\nCurrencies Present:")
67 print(df["UnitPrice"].str.extract(r"([A-Za-z]+)$")[0].unique())
68 # Identify inconsistent city names
69 print("\nCity Names:")
70 print(df["City"].unique())
71 # Convert dates and detect invalid dates
72 df["ParsedDate"] = pd.to_datetime(
73     df["PurchaseDate"],
74     dayfirst=True,
75     errors="coerce"
76 )
77 print("\nInvalid Purchase Dates:")
78 print(df[df["ParsedDate"].isna()][
79     ["CustomerID", "PurchaseDate"]
80 ])
81 # =====
82 # TASK 2: Design a cleaning strategy for each issue.
83 # =====
84 print("\n\nTASK 2: CLEANING THE DATA")
85 cleaned_df = df.copy()
86 # Remove duplicate transactions
87 cleaned_df = cleaned_df.drop_duplicates()

```

```

88 # Remove records with missing CustomerID
89 cleaned_df = cleaned_df.dropna(
90     subset=["CustomerID"]
91 )
92 # Remove transactions with negative quantity
93 cleaned_df = cleaned_df[
94     cleaned_df["Quantity"] >= 0
95 ]
96 # Replace unrealistic ages with missing values
97 cleaned_df.loc[
98     (cleaned_df["Age"] < 1) |
99     (cleaned_df["Age"] > 120),
100     "Age"
101 ] = np.nan
102 # Fill missing age using median age
103 median_age = cleaned_df["Age"].median()
104 cleaned_df["Age"] = cleaned_df["Age"].fillna(
105     median_age
106 )
107 # Remove records with invalid purchase dates
108 cleaned_df = cleaned_df.dropna(
109     subset=["ParsedDate"]
110 )
111 # Standardize city names
112 city_mapping = {
113     "Chennai": "Chennai",
114     "chennai": "Chennai",
115     "CHENNAI": "Chennai",
116     "Trichy": "Tiruchirappalli",

```

```

117     "Tiruchirappalli": "Tiruchirappalli"
118 }
119 cleaned_df["City"] = cleaned_df["City"].replace(
120     city_mapping
121 )
122 print("\nCleaned Data:")
123 print(cleaned_df)
124 # =====
125 # TASK 3: Perform data standardization and transformation.
126 # =====
127 print("\n\nTASK 3: DATA STANDARDIZATION AND TRANSFORMATION")
128 # Extract numerical value from UnitPrice
129 cleaned_df["Price"] = (
130     cleaned_df["UnitPrice"]
131     .str.extract(r"([\d.]+)")
132     .astype(float)
133 )
134 # Extract currency
135 cleaned_df["Currency"] = (
136     cleaned_df["UnitPrice"]
137     .str.extract(r"([A-Za-z]+)$")
138 )
139 # Exchange rates to INR
140 exchange_rates = {
141     "INR": 1,
142     "USD": 83,
143     "EUR": 90
144 }
145 # Convert all prices to INR
146
147 # Convert all prices to INR
148 cleaned_df["UnitPrice_INR"] = (
149     cleaned_df["Price"] *
150     cleaned_df["Currency"].map(exchange_rates)
151 )
152 # Calculate revenue
153 cleaned_df["Revenue_INR"] = (
154     cleaned_df["Quantity"] *
155     cleaned_df["UnitPrice_INR"]
156 )
157 # Standardize purchase date
158 cleaned_df["PurchaseDate"] = cleaned_df["ParsedDate"]
159 # Remove temporary column
160 cleaned_df = cleaned_df.drop(
161     columns=["ParsedDate"]
162 )
163 print("\nTransformed Data:")
164 print(cleaned_df)
165 # =====
166 # TASK 4: Analyze how cleaning affects total revenue calculations.
167 # =====
168 print("\n\nTASK 4: REVENUE ANALYSIS")
169 # Calculate revenue from original dataset
170 original_df = df.copy()
171 original_df["Price"] = (
172     original_df["UnitPrice"]
173     .str.extract(r"([\d.]+)")
174     .astype(float)
175 )

```

```

174 original_df["Currency"] = (
175     original_df["UnitPrice"]
176     .str.extract(r"([A-Za-z]+)$")
177 )
178 original_df["UnitPrice_INR"] = (
179     original_df["Price"] *
180     original_df["Currency"].map(exchange_rates)
181 )
182 original_df["Revenue_INR"] = (
183     original_df["Quantity"] *
184     original_df["UnitPrice_INR"]
185 )
186 revenue_before = original_df[
187     "Revenue_INR"
188 ].sum()
189 # Calculate revenue after cleaning
190 revenue_after = cleaned_df[
191     "Revenue_INR"
192 ].sum()
193 print("Total Revenue Before Cleaning:",
194     revenue_before, "INR")
195 print("Total Revenue After Cleaning:",
196     revenue_after, "INR")
197 print("Difference in Revenue:",
198     revenue_before - revenue_after, "INR")
199 # Revenue by product category
200 print("\nRevenue by Product Category:")
201 category_revenue = (
202     cleaned_df
203     .groupby("ProductCategory")["Revenue_INR"]
204     .sum()
205     .sort_values(ascending=False)
206 )
207 print(category_revenue)
208 # Top customers
209 print("\nTop Customers:")
210 customer_revenue = (
211     cleaned_df
212     .groupby("CustomerName")["Revenue_INR"]
213     .sum()
214     .sort_values(ascending=False)
215 )
216 print(customer_revenue)
217 # Calculate percentage change in revenue
218 percentage_change = (
219     (revenue_after - revenue_before)
220     / revenue_before
221 ) * 100
222 print(
223     "Percentage Change in Revenue:",
224     round(percentage_change, 2),
225     "%"
226 )
227 # =====
228 # TASK 5: Generate a report discussing business risks if the
229 # data were not cleaned.
230 # =====
231 print("\n\nTASK 5: BUSINESS RISK REPORT")

```

```

232 print("""
233 BUSINESS RISK REPORT
234 =====
235 1. Incorrect Revenue:
236 | Different currencies, negative quantities and duplicate
237 | transactions can result in incorrect revenue calculations.
238 2. Incorrect Customer Analysis:
239 | Missing CustomerIDs and duplicate transactions can make
240 | customer spending appear higher or lower than the actual value.
241 3. Incorrect Product Performance:
242 | Poor-quality transactions can incorrectly identify the
243 | best-selling or most profitable products.
244 4. Incorrect Business Decisions:
245 | Management may make incorrect decisions about pricing,
246 | inventory, marketing and sales strategies.
247 5. Poor Customer Segmentation:
248 | Missing or incorrect customer information can result in
249 | incorrect customer segmentation.
250 6. Recommended Controls:
251 | - Make CustomerID mandatory.
252 | - Prevent duplicate transactions.
253 | - Validate quantity values.
254 | - Validate customer age.
255 | - Standardize currency to INR.
256 | - Validate purchase dates.
257 | - Maintain standardized city names.
258 | - Perform regular data-quality checks.
259 """)
260 # =====

```

```

    Poor-quality transactions can incorrectly identify the
    best-selling or most profitable products.
4. Incorrect Business Decisions:
    Management may make incorrect decisions about pricing,
    inventory, marketing and sales strategies.
5. Poor Customer Segmentation:
    Missing or incorrect customer information can result in
    incorrect customer segmentation.
6. Recommended Controls:
    - Make CustomerID mandatory.
    - Prevent duplicate transactions.
    - Validate quantity values.
    - Validate customer age.
    - Standardize currency to INR.
    - Validate purchase dates.
    - Maintain standardized city names.
    - Perform regular data-quality checks.
"""

```

```

# =====
# DISPLAY FINAL CLEAN DATA
# =====
print("\n\nFINAL CLEAN E-COMMERCE DATA")
print(cleaned_df.to_string(index=False))

```

OUTPUT:

ORIGINAL DATASET:

0	C101	Raj Kumar	Male	28.0	Electronics	2	15000	INR	10-01-2025	Chennai
1	C102	Priya S	Female	35.0	Fashion	3	1200	INR	11-01-2025	chennai
2	C103	Arun V	Male	150.0	Electronics	1	300	USD	12-01-2025	CHENNAI
3	C104	Kavya R	Female	0.0	Grocery	-2	800	INR	15-01-2025	Coimbatore
4	C105	Rahul P	Male	42.0	Fashion	5	2500	INR	18-01-2025	Madurai
5	C105	Rahul P	Male	42.0	Fashion	5	2500	INR	18-01-2025	Madurai
6	NaN	Sneha M	Female	30.0	Electronics	2	450	USD	05-02-2025	Trichy
7	C107	Deepak N	Male	NaN	Grocery	3	600	INR	07-02-2025	Tiruchirappalli

TASK:1

Missing Values:

CustomerID	1
CustomerName	0
Gender	0
Age	1
ProductCategory	0
Quantity	0
UnitPrice	0

Duplicate Transactions:

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	
4	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	18-01-2025	Madurai
5	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	18-01-2025	Madurai

Invalid Ages:

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	
2	C103	Arun V	Male	150.0	Electronics	1	300 USD	12-01-2025	CHENNAI
3	C104	Kavya R	Female	0.0	Grocery	-2	800 INR	15-01-2025	Coimbatore

Negative Quantities:

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	
3	C104	Kavya R	Female	0.0	Grocery	-2	800 INR	15-01-2025	Coimbatore

Currencies Present:

<StringArray>

['INR', 'USD', 'EUR']

Length: 3, dtype: str

City Names:

<StringArray>

['Chennai', 'chennai', 'CHENNAI', 'Coimbatore', 'Madurai', 'Trichy', 'Tiruchirappalli', 'Salem']

Length: 8, dtype: str

Invalid Purchase Dates:

CustomerID	PurchaseDate
8	C108 2025-13-01

TASK 2: CLEANING THE DATA

Cleaned Data:

	CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	ParsedDate
0	C101	Raj Kumar	Male	28.0	Electronics	2	15000 INR	2025-01-10	Chennai	2025-01-10
1	C102	Priya S	Female	35.0	Fashion	3	1200 INR	2025-01-11	Chennai	2025-01-11
2	C103	Arun V	Male	31.0	Electronics	1	300 USD	2025-01-12	Chennai	2025-01-12
4	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	2025-01-18	Madurai	2025-01-18
7	C107	Deepak N	Male	31.0	Grocery	3	600 INR	2025-02-07	Tiruchirappalli	2025-02-07
9	C109	John D	Male	31.0	Electronics	4	250 EUR	2025-02-15	Chennai	2025-02-15

TASK 3: DATA STANDARDIZATION AND TRANSFORMATION

Transformed Data:

	CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	Price	Currency	UnitPrice_INR	Revenue_INR
0	C101	Raj Kumar	Male	28.0	Electronics	2	15000 INR	2025-01-10	Chennai	15000.0	INR	15000.0	30000.0
1	C102	Priya S	Female	35.0	Fashion	3	1200 INR	2025-01-11	Chennai	1200.0	INR	1200.0	3600.0
2	C103	Arun V	Male	31.0	Electronics	1	300 USD	2025-01-12	Chennai	300.0	USD	24900.0	24900.0
4	C105	Rahul P	Male	42.0	Fashion	5	2500 INR	2025-01-18	Madurai	2500.0	INR	2500.0	12500.0
7	C107	Deepak N	Male	31.0	Grocery	3	600 INR	2025-02-07	Tiruchirappalli	600.0	INR	600.0	1800.0
9	C109	John D	Male	31.0	Electronics	4	250 EUR	2025-02-15	Chennai	250.0	EUR	22500.0	90000.0

TASK 4: REVENUE ANALYSIS

```
Total Revenue Before Cleaning: 250200.0 INR
Total Revenue After Cleaning: 162800.0 INR
Difference in Revenue: 87400.0 INR
```

```
Revenue by Product Category:
ProductCategory
Electronics      144900.0
Fashion          16100.0
Grocery          1800.0
Name: Revenue_INR, dtype: float64
```

```
Top Customers:
CustomerName
John D      90000.0
Raj Kumar   30000.0
Arun V      24900.0
Rahul P     12500.0
Priya S      3600.0
Deepak N    1800.0
```

TASK 5: BUSINESS RISK REPORT

- 1. Incorrect Revenue:
Different currencies, negative quantities and duplicate transactions can result in incorrect revenue calculations.
- 2. Incorrect Customer Analysis:
Missing CustomerIDs and duplicate transactions can make customer spending appear higher or lower than the actual value.
- 3. Incorrect Product Performance:
Poor-quality transactions can incorrectly identify the best-selling or most profitable products.
- 4. Incorrect Business Decisions:
Management may make incorrect decisions about pricing, inventory, marketing and sales strategies.
- 5. Poor Customer Segmentation:
Missing or incorrect customer information can result in incorrect customer segmentation.

FULL CLEANED DATA:

CustomerID	CustomerName	Gender	Age	ProductCategory	Quantity	UnitPrice	PurchaseDate	City	Price	Currency	UnitPrice_INR	Revenue_INR
C101	Raj Kumar	Male	28.0	Electronics	2	15000	INR 2025-01-10	Chennai	15000.0	INR	15000.0	30000.0
C102	Priya S	Female	35.0	Fashion	3	1200	INR 2025-01-11	Chennai	1200.0	INR	1200.0	3600.0
C103	Arun V	Male	31.0	Electronics	1	300	USD 2025-01-12	Chennai	300.0	USD	24900.0	24900.0
C105	Rahul P	Male	42.0	Fashion	5	2500	INR 2025-01-18	Madurai	2500.0	INR	2500.0	12500.0
C107	Deepak N	Male	31.0	Grocery	3	600	INR 2025-02-07	Tiruchirappalli	600.0	INR	600.0	1800.0
C109	John D	Male	31.0	Electronics	4	250	EUR 2025-02-15	Chennai	250.0	EUR	22500.0	90000.0

Question 2: Hospital Patient Records Dataset

A hospital wants to analyze patient admissions and treatment effectiveness. Data were entered manually by different departments, leading to inconsistencies.

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
P1001	Ravi Kumar	45	M	10-01-2025	15-01-2025	Diabetes	25000	D101
P1002	Priya S	38	F	12-01-2025	11-01-2025	Hypertension	18000	D102
P1003	Arun V	125	M	13-01-2025	20-01-2025	diabetes	22000	D101
NULL	Kavya R	29	F	15-01-2025	17-01-2025	Diabtes	15000	D103
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1005	Rahul P	51	M	18-01-2025	25-01-2025	Asthma	NULL	D104
P1006	Sneha M	34	F	15-01-2025	18-01-2025	Hypertension	12000	D102
P1007	Deepak N		M	01-02-2025	05-02-2025	Asthma	10000	D104
P1008	Meena K	26	F	10-02-2025	15-02-2025	HYPERTENSION	13500	D102
P1009	John D	40	M	12-02-2025	18-02-2025	Diabetes	20000	D101

Data Issues

- Missing patient IDs
- Duplicate patient records
- Discharge dates before admission dates
- Null treatment costs
- Inconsistent diagnosis names
- Multiple formats of dates
- Unrealistic patient ages (>120)

Tasks

1. Detect all inconsistencies and anomalies. (5 Marks)
2. Develop rules for handling missing and invalid values. (5 Marks)
3. Create a process to standardize diagnosis names. (5 Marks)
4. Evaluate how cleaning changes average treatment cost and hospital stay duration. (5 Marks)
5. Recommend data governance measures to prevent future errors. (5 Marks)

```

1  import pandas as pd
2  import numpy as np
3  # =====
4  # HOSPITAL PATIENT RECORDS DATASET
5  # =====
6  # Create the dataset
7  data = {
8      "PatientID": [
9          "P1001", "P1002", "P1003", None, "P1005",
10         "P1005", "P1006", "P1007", "P1008", "P1009"
11     ],
12     "Name": [
13         "Ravi Kumar", "Priya S", "Arun V", "Kavya R",
14         "Rahul P", "Rahul P", "Sneha M", "Deepak N",
15         "Meena K", "John D"
16     ],
17     "Age": [
18         45, 38, 125, 29, 51,
19         51, 34, np.nan, 26, 40
20     ],
21     "Gender": [
22         "M", "F", "M", "F", "M",
23         "M", "F", "M", "F", "M"
24     ],
25     "AdmissionDate": [
26         "10-01-2025", "12-01-2025", "13-01-2025",
27         "15-01-2025", "18-01-2025", "18-01-2025",
28         "15-01-2025", "01-02-2025", "10-02-2025",
29         "12-02-2025"
30     ],
31     "DischargeDate": [
32         "15-01-2025", "11-01-2025", "20-01-2025",
33         "17-01-2025", "25-01-2025", "25-01-2025",
34         "18-01-2025", "05-02-2025", "15-02-2025",
35         "18-02-2025"
36     ],
37     "Diagnosis": [
38         "Diabetes", "Hypertension", "diabetes", "Diabtes",
39         "Asthma", "Asthma", "Hypertension", "Asthma",
40         "HYPERTENSION", "Diabetes"
41     ],
42     "TreatmentCost": [
43         25000, 18000, 22000, 15000, np.nan,
44         np.nan, 12000, 10000, 13500, 20000
45     ],
46     "DoctorID": [
47         "D101", "D102", "D101", "D103", "D104",
48         "D104", "D102", "D104", "D102", "D101"
49     ]
50 }
51 df = pd.DataFrame(data)
52 print("ORIGINAL HOSPITAL DATA")
53 print(df)
54 # =====
55 # TASK 1: Detect all inconsistencies and anomalies.
56 # =====
57 print("\n\nTASK 1: DETECT INCONSISTENCIES AND ANOMALIES")
58 # Check missing values
59 print("\nMissing Values:")
60 print(df.isnull().sum())
61 # Check duplicate records
62 print("\nDuplicate Records:")
63 print(df[df.duplicated(keep=False)])
64 # Check unrealistic ages
65 print("\nUnrealistic Ages:")
66 print(
67     df[
68         (df["Age"] < 1) |
69         (df["Age"] > 120)
70     ]
71 )

```

```

72 # Convert dates
73 df["AdmissionParsed"] = pd.to_datetime(
74     df["AdmissionDate"],
75     dayfirst=True,
76     errors="coerce"
77 )
78 df["DischargeParsed"] = pd.to_datetime(
79     df["DischargeDate"],
80     dayfirst=True,
81     errors="coerce"
82 )
83 # Check discharge dates before admission dates
84 print("\nDischarge Before Admission:")
85 print(
86     df[
87         df["DischargeParsed"] <
88         df["AdmissionParsed"]

```

```

89     ]
90     ["PatientID", "AdmissionDate", "DischargeDate"]
91 ]
92 )
93 # Check missing treatment costs
94 print("\nMissing Treatment Costs:")
95 print(
96     df[df["TreatmentCost"].isnull()]
97 )
98 # Check diagnosis inconsistencies
99 print("\nDiagnosis Values:")
100 print(df["Diagnosis"].unique())
101 # =====
102 # TASK 2: Develop rules for handling missing and invalid values.
103 # =====
104 print("\n\nTASK 2: HANDLE MISSING AND INVALID VALUES")
105 cleaned_df = df.copy()
106 # Remove duplicate patient records

```

```

106 # Remove duplicate patient records
107 cleaned_df = cleaned_df.drop_duplicates()
108 # Remove records without PatientID
109 cleaned_df = cleaned_df.dropna(
110     subset=["PatientID"]
111 )
112 # Replace unrealistic ages with missing values
113 cleaned_df.loc[
114     (cleaned_df["Age"] < 1) |
115     (cleaned_df["Age"] > 120),
116     "Age"
117 ] = np.nan
118 # Fill missing age using median
119 median_age = cleaned_df["Age"].median()
120 cleaned_df["Age"] = (
121     cleaned_df["Age"]
122     .fillna(median_age)
123 )

```

```

124 # Fill missing treatment cost using median
125 median_cost = (
126     cleaned_df["TreatmentCost"].median()
127 )
128 cleaned_df["TreatmentCost"] = (
129     cleaned_df["TreatmentCost"]
130     .fillna(median_cost)
131 )
132 # Remove records where discharge is before admission
133 cleaned_df = cleaned_df[
134     cleaned_df["DischargeParsed"] >=
135     cleaned_df["AdmissionParsed"]
136 ]
137 print("\nCleaned Data:")
138 print(cleaned_df)

```

```

139 # =====
140 # TASK 3: Create a process to standardize diagnosis names.
141 # =====
142 print("\n\nTASK 3: STANDARDIZE DIAGNOSIS NAMES")
143 # Convert diagnosis names to lowercase
144 cleaned_df["Diagnosis"] = (
145     cleaned_df["Diagnosis"]
146     .str.strip()
147     .str.lower()
148 )
149 # Standardize diagnosis names
150 diagnosis_mapping = {
151     "diabetes": "Diabetes",
152     "diabtes": "Diabetes",
153     "hypertension": "Hypertension",
154     "asthma": "Asthma"
155 }
156 cleaned_df["Diagnosis"] = (
157     cleaned_df["Diagnosis"]
158     .replace(diagnosis_mapping)
159 )
160 # Standardize date columns
161 cleaned_df["AdmissionDate"] = (
162     cleaned_df["AdmissionParsed"]
163 )
164 cleaned_df["DischargeDate"] = (
165     cleaned_df["DischargeParsed"]
166 )
167 # Calculate hospital stay duration
168 cleaned_df["StayDuration"] = (
169     cleaned_df["DischargeDate"] -
170     cleaned_df["AdmissionDate"]
171 ).dt.days
172 print("\n\nStandardized Data:")
173 print(cleaned_df)
174 # =====
175 # TASK 4: Evaluate how cleaning changes average treatment cost
176 #         and hospital stay duration.
177 # =====
178 print("\n\nTASK 4: ANALYZE TREATMENT COST AND HOSPITAL STAY")
179 # Calculate stay duration for original dataset
180 df["StayDuration"] = (
181     df["DischargeParsed"] -
182     df["AdmissionParsed"]
183 ).dt.days
184 # Average treatment cost before cleaning
185 average_cost_before = (
186     df["TreatmentCost"].mean()
187 )
188 # Average valid hospital stay before cleaning
189 average_stay_before = (
190     df[
191         df["StayDuration"] >= 0
192     ]
193 )
194 # Average treatment cost after cleaning
195 average_cost_after = (
196     cleaned_df["TreatmentCost"].mean()
197 )
198 # Average hospital stay after cleaning
199 average_stay_after = (
200     cleaned_df["StayDuration"].mean()
201 )
202 print(
203     "Average Treatment Cost Before Cleaning:",
204     round(average_cost_before, 2)
205 )
206 print(
207     "Average Treatment Cost After Cleaning:",
208     round(average_cost_after, 2)
209 )
210 print(
211     "\nAverage Hospital Stay Before Cleaning:",

```

```

211     "\nAverage Hospital Stay Before Cleaning:",
212     round(average_stay_before, 2),
213     "days"
214 )
215 print(
216     "Average Hospital Stay After Cleaning:",
217     round(average_stay_after, 2),
218     "days"
219 )
220 print(
221     "\nChange in Average Treatment Cost:",
222     round(
223         average_cost_after -
224         average_cost_before,
225         2
226     )
227 )
228 print(
229     "Change in Average Hospital Stay:",
230     round(
231         average_stay_after -
232         average_stay_before,
233         2
234     ),
235     "days")
236 # Average treatment cost by diagnosis
237 print("\nAverage Treatment Cost by Diagnosis:")
238 print(
239     cleaned_df
240     .groupby("Diagnosis")["TreatmentCost"]
241     .mean())
242 # Average hospital stay by diagnosis
243 print("\nAverage Hospital Stay by Diagnosis:")
244 print(
245     cleaned_df
246     .groupby("Diagnosis")["StayDuration"]
247 )
248
249 # =====
250 # TASK 5: Recommend data governance measures to prevent future
251 #     errors.
252 # =====
253 print("\n\nTASK 5: DATA GOVERNANCE RECOMMENDATIONS")
254 print("""
255 DATA GOVERNANCE RECOMMENDATIONS
256 =====
257 1. PatientID should be mandatory and unique.
258 2. The hospital system should reject unrealistic ages.
259 3. AdmissionDate and DischargeDate should be validated
260    automatically.
261 4. DischargeDate should not be earlier than AdmissionDate.
262 5. Diagnosis should be selected from a predefined list.
263 6. TreatmentCost should be a mandatory numeric field.
264 7. Duplicate patient records should be automatically detected.
265 8. Mandatory fields should be enforced during data entry.
266 9. Regular data-quality audits should be performed.

```

```

266 9. Regular data-quality audits should be performed.
267 10. Only authorized staff should be allowed to modify
268 | patient records.
269 11. Automated validation rules should be implemented.
270 12. A data-quality dashboard should be used to monitor
271 | missing values, duplicates and invalid records.
272 """)
273 # =====
274 # DISPLAY FINAL CLEAN DATA
275 # =====
276 print("\n\nFINAL CLEAN HOSPITAL DATA")
277 print(cleaned_df.to_string(index=False))

```

OUTPUT:

ORIGINAL DATASET:

0	P1001	Ravi Kumar	45.0	M	10-01-2025	15-01-2025	Diabetes	25000.0	D101
1	P1002	Priya S	38.0	F	12-01-2025	11-01-2025	Hypertension	18000.0	D102
2	P1003	Arun V	125.0	M	13-01-2025	20-01-2025	diabetes	22000.0	D101
3	NaN	Kavya R	29.0	F	15-01-2025	17-01-2025	Diabtes	15000.0	D103
4	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104
5	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104
6	P1006	Sneha M	34.0	F	15-01-2025	18-01-2025	Hypertension	12000.0	D102
7	P1007	Deepak N	NaN	M	01-02-2025	05-02-2025	Asthma	10000.0	D104

TASK 1: DETECT INCONSISTENCIES AND ANOMALIES

Missing Values:

```

PatientID      1
Name           0
Age            1
Gender         0
AdmissionDate  0
DischargeDate  0
Diagnosis      0

```

Duplicate Records:

	PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
4	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104
5	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104

Unrealistic Ages:

	PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID
2	P1003	Arun V	125.0	M	13-01-2025	20-01-2025	diabetes	22000.0	D101

Discharge Before Admission:

	PatientID	AdmissionDate	DischargeDate
1	P1002	12-01-2025	11-01-2025

Missing Treatment Costs:

	PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID	AdmissionParsed	DischargeParsed
4	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104	2025-01-18	2025-01-25
5	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	NaN	D104	2025-01-18	2025-01-25

Diagnosis Values:
<StringArray>
['Diabetes', 'Hypertension', 'diabetes', 'Diabetes', 'Asthma', 'HYPERTENSION']
Length: 6, dtype: str

TASK 2: HANDLE MISSING AND INVALID VALUES
CLEANED DATA:

	PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID	AdmissionParsed	DischargeParsed
0	P1001	Ravi Kumar	45.0	M	10-01-2025	15-01-2025	Diabetes	25000.0	D101	2025-01-10	2025-01-15
2	P1003	Arun V	39.0	M	13-01-2025	20-01-2025	diabetes	22000.0	D101	2025-01-13	2025-01-20
4	P1005	Rahul P	51.0	M	18-01-2025	25-01-2025	Asthma	18000.0	D104	2025-01-18	2025-01-25
6	P1006	Sneha M	34.0	F	15-01-2025	18-01-2025	Hypertension	12000.0	D102	2025-01-15	2025-01-18
7	P1007	Deepak N	39.0	M	01-02-2025	05-02-2025	Asthma	10000.0	D104	2025-02-01	2025-02-05
8	P1008	Meena K	26.0	F	10-02-2025	15-02-2025	HYPERTENSION	13500.0	D102	2025-02-10	2025-02-15
9	P1009	John D	40.0	M	12-02-2025	18-02-2025	Diabetes	20000.0	D101	2025-02-12	2025-02-18

TASK 3: STANDARDIZE DIAGNOSIS NAMES

Standardized Data:

	PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID	AdmissionParsed	DischargeParsed	StayDuration
0	P1001	Ravi Kumar	45.0	M	2025-01-10	2025-01-15	Diabetes	25000.0	D101	2025-01-10	2025-01-15	5
2	P1003	Arun V	39.0	M	2025-01-13	2025-01-20	Diabetes	22000.0	D101	2025-01-13	2025-01-20	7
4	P1005	Rahul P	51.0	M	2025-01-18	2025-01-25	Asthma	18000.0	D104	2025-01-18	2025-01-25	7
6	P1006	Sneha M	34.0	F	2025-01-15	2025-01-18	Hypertension	12000.0	D102	2025-01-15	2025-01-18	3
7	P1007	Deepak N	39.0	M	2025-02-01	2025-02-05	Asthma	10000.0	D104	2025-02-01	2025-02-05	4
8	P1008	Meena K	26.0	F	2025-02-10	2025-02-15	Hypertension	13500.0	D102	2025-02-10	2025-02-15	5
9	P1009	John D	40.0	M	2025-02-12	2025-02-18	Diabetes	20000.0	D101	2025-02-12	2025-02-18	6

Average Treatment Cost Before Cleaning: 16937.5
Average Treatment Cost After Cleaning: 17214.29

Average Hospital Stay Before Cleaning: 5.11 days
Average Hospital Stay After Cleaning: 5.29 days

Change in Average Treatment Cost: 276.79
Change in Average Hospital Stay: 0.17 days

Average Treatment Cost by Diagnosis:
Diagnosis
Asthma 14000.000000
Diabetes 22333.333333
Hypertension 12750.000000

DATA GOVERNANCE RECOMMENDATIONS
=====

- 1. PatientID should be mandatory and unique.
- 2. The hospital system should reject unrealistic ages.

- 3. AdmissionDate and DischargeDate should be validated automatically.
- 4. DischargeDate should not be earlier than AdmissionDate.
- 5. Diagnosis should be selected from a predefined list.

- 6. TreatmentCost should be a mandatory numeric field.
- 7. Duplicate patient records should be automatically detected.
- 8. Mandatory fields should be enforced during data entry.
- 9. Regular data-quality audits should be performed.

10. Only authorized staff should be allowed to modify patient records.

11. Automated validation rules should be implemented.

12. A data-quality dashboard should be used to monitor missing values, duplicates and invalid records.

FULL CLEANED DATA:

PatientID	Name	Age	Gender	AdmissionDate	DischargeDate	Diagnosis	TreatmentCost	DoctorID	AdmissionParsed	DischargeParsed	StayDuration
P1001	Ravi Kumar	45.0	M	2025-01-10	2025-01-15	Diabetes	25000.0	D101	2025-01-10	2025-01-15	5
P1003	Arun V	39.0	M	2025-01-13	2025-01-20	Diabetes	22000.0	D101	2025-01-13	2025-01-20	7
P1005	Rahul P	51.0	M	2025-01-18	2025-01-25	Asthma	18000.0	D104	2025-01-18	2025-01-25	7
P1006	Sneha M	34.0	F	2025-01-15	2025-01-18	Hypertension	12000.0	D102	2025-01-15	2025-01-18	3
P1007	Deepak N	39.0	M	2025-02-01	2025-02-05	Asthma	10000.0	D104	2025-02-01	2025-02-05	4
P1008	Meena K	26.0	F	2025-02-10	2025-02-15	Hypertension	13500.0	D102	2025-02-10	2025-02-15	5
P1009	John D	40.0	M	2025-02-12	2025-02-18	Diabetes	20000.0	D101	2025-02-12	2025-02-18	6